

N° attribué par la bibliothèque

/ \_ / \_ / \_ / \_ / \_ / \_ /

## RAPPORT DE STAGE ACADÉMIQUE

**Domaine :** Sciences et Technologies

**Mention :** Sciences et Techniques

**Spécialité :** Mathématiques-Informatique

**ÉTUDE ET IMPLÉMENTATION FROM SCRATCH DE LA RÉGRESSION LOGISTIQUE**

Présenté par : **ADJAGBENON** Elisée

Soutenu le : ..... / ..... /20 .....

**Dirigé par**

| Tuteur                                                                       | Maître de stage                                                                                                   |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Dr. AGUEMON Uriel</b><br><br>Enseignant-chercheur à la FAST de Natitingou | <b>Mr. OROU BOUYAGUI Bio Mourou</b><br><br>Responsable du laboratoire de l'Intelligence Artificielle (IA) du CAEB |

**Année académique : 2025 – 2026**

# Contents

|                                                                                                                                                                                      |           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| RÉSUMÉ . . . . .                                                                                                                                                                     | viii      |
| SUMMARY . . . . .                                                                                                                                                                    | ix        |
| DÉDICACES . . . . .                                                                                                                                                                  | x         |
| REMERCIEMENTS . . . . .                                                                                                                                                              | x         |
| INTRODUCTION GÉNÉRALE . . . . .                                                                                                                                                      | 1         |
| <b>0 Présentation de notre structure de formation, de la structure d'accueil du stage, et bilan du séjour immersif et des tâches effectuées dans l'organisme d'accueil . . . . .</b> | <b>3</b>  |
| 0.1 Présentation de notre structure de formation . . . . .                                                                                                                           | 4         |
| 0.1.1 Situation géographique, création et évolution . . . . .                                                                                                                        | 4         |
| 0.1.2 Personnels administratifs, enseignants . . . . .                                                                                                                               | 4         |
| 0.1.3 Infrastructures, mobiliers et matériels didactiques . . . . .                                                                                                                  | 4         |
| 0.2 Présentation du lieu de stage . . . . .                                                                                                                                          | 5         |
| 0.2.1 Situation géographique, création et évolution . . . . .                                                                                                                        | 5         |
| 0.2.2 Personnels administratifs, enseignants, stagiaires . . . . .                                                                                                                   | 6         |
| 0.2.3 Infrastructures, mobiliers et matériels didactiques . . . . .                                                                                                                  | 6         |
| 0.3 Organisation administrative et pédagogique . . . . .                                                                                                                             | 7         |
| 0.3.1 Club IA et différents partenaires du CAEB . . . . .                                                                                                                            | 7         |
| 0.4 Programme de formation ou stage . . . . .                                                                                                                                        | 8         |
| 0.4.1 Phase théorique . . . . .                                                                                                                                                      | 8         |
| 0.4.2 Phase pratique . . . . .                                                                                                                                                       | 9         |
| 0.5 Suivi évaluation . . . . .                                                                                                                                                       | 9         |
| 0.6 Difficultés rencontrées et remédiations apportées . . . . .                                                                                                                      | 10        |
| 0.7 Enseignements tirés ou leçons apprises . . . . .                                                                                                                                 | 10        |
| 0.8 Impressions générales et suggestions . . . . .                                                                                                                                   | 10        |
| <b>1 Cadre théorique et état de l'art . . . . .</b>                                                                                                                                  | <b>11</b> |
| 1.1 Généralités sur l'apprentissage automatique supervisé . . . . .                                                                                                                  | 11        |
| 1.2 La régression logistique : origines et principes . . . . .                                                                                                                       | 11        |
| 1.2.1 Domaines d'application . . . . .                                                                                                                                               | 12        |
| 1.3 Comparaison avec d'autres méthodes de classification . . . . .                                                                                                                   | 12        |

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 1.3.1    | Notion de frontière de décision linéaire . . . . .                    | 13        |
| 1.4      | Travaux et outils existants . . . . .                                 | 13        |
| <b>2</b> | <b>Fondements mathématiques de la régression logistique . . . . .</b> | <b>15</b> |
| 2.1      | De la régression linéaire à la régression logistique . . . . .        | 15        |
| 2.2      | La fonction sigmoïde . . . . .                                        | 15        |
| 2.3      | Fonction de coût : l'entropie croisée . . . . .                       | 16        |
| 2.4      | Optimisation par descente de gradient . . . . .                       | 17        |
| 2.4.1    | Calcul du gradient . . . . .                                          | 17        |
| 2.4.2    | Règle de mise à jour . . . . .                                        | 18        |
| 2.4.3    | Critères d'arrêt . . . . .                                            | 18        |
| 2.4.4    | Variantes de la descente de gradient . . . . .                        | 19        |
| 2.5      | Régularisation . . . . .                                              | 19        |
| 2.5.1    | Régularisation L2 (Ridge) . . . . .                                   | 19        |
| 2.5.2    | Régularisation L1 (Lasso) . . . . .                                   | 19        |
| 2.6      | Extension au cas multiclasse . . . . .                                | 20        |
| <b>3</b> | <b>Conception et implémentation from scratch . . . . .</b>            | <b>21</b> |
| 3.1      | Environnement et outils utilisés . . . . .                            | 21        |
| 3.2      | Architecture du code . . . . .                                        | 21        |
| 3.3      | Implémentation de l'algorithme . . . . .                              | 22        |
| 3.3.1    | Fonction sigmoïde . . . . .                                           | 22        |
| 3.3.2    | Boucle d'entraînement . . . . .                                       | 22        |
| 3.3.3    | Prédiction . . . . .                                                  | 23        |
| 3.4      | Prétraitement des données . . . . .                                   | 23        |
| 3.5      | Choix des hyperparamètres . . . . .                                   | 24        |
| 3.6      | Complexité algorithmique . . . . .                                    | 24        |
| 3.7      | Validation de l'implémentation . . . . .                              | 25        |
| 3.8      | Reproductibilité . . . . .                                            | 25        |
| <b>4</b> | <b>Expérimentation, résultats et discussion . . . . .</b>             | <b>26</b> |
| 4.1      | Présentation du jeu de données . . . . .                              | 26        |
| 4.2      | Métriques d'évaluation . . . . .                                      | 26        |
| 4.3      | Résultats obtenus . . . . .                                           | 27        |
| 4.3.1    | Convergence de l'entraînement . . . . .                               | 27        |
| 4.3.2    | Performances sur le jeu de test . . . . .                             | 27        |
| 4.3.3    | Matrice de confusion . . . . .                                        | 27        |
| 4.3.4    | Courbe ROC . . . . .                                                  | 28        |

---

|       |                                                 |    |
|-------|-------------------------------------------------|----|
| 4.4   | Comparaison avec scikit-learn . . . . .         | 28 |
| 4.4.1 | Comparaison des temps d'exécution . . . . .     | 29 |
| 4.5   | Analyse de l'importance des variables . . . . . | 29 |
| 4.6   | Sensibilité à la régularisation . . . . .       | 30 |
| 4.6.1 | Interprétation des résultats . . . . .          | 31 |
| 4.6.2 | Difficultés rencontrées . . . . .               | 31 |
| 4.6.3 | Limites du travail . . . . .                    | 31 |
| 4.6.4 | Perspectives . . . . .                          | 32 |
|       | CONCLUSION GÉNÉRALE . . . . .                   | 33 |
|       | BIBLIOGRAPHIE . . . . .                         | 34 |

# List of Figures

|     |                                                                                          |    |
|-----|------------------------------------------------------------------------------------------|----|
| 2.1 | Représentation de la fonction sigmoïde $\sigma(z)$ . . . . .                             | 16 |
| 2.2 | Effet du taux d'apprentissage $\alpha$ sur la convergence de la fonction de coût . . . . | 18 |
| 4.1 | Évolution de la fonction de coût pendant l'entraînement (3000 itérations) . . . .        | 27 |
| 4.2 | Matrice de confusion sur le jeu de test . . . . .                                        | 28 |
| 4.3 | Courbe ROC de l'implémentation from scratch sur le jeu de test . . . . .                 | 28 |
| 4.4 | Dix variables les plus influentes selon les coefficients appris par le modèle . . . .    | 30 |

# List of Tables

|     |                                                                                  |    |
|-----|----------------------------------------------------------------------------------|----|
| 1   | Personnels administratifs et enseignants de la FAST Natitingou . . . . .         | 5  |
| 2   | Personnels du CAEB Natitingou . . . . .                                          | 6  |
| 1.1 | Comparaison de la régression logistique avec d'autres méthodes de classification | 13 |
| 4.1 | Performances de l'implémentation from scratch sur le jeu de test . . . . .       | 27 |
| 4.2 | Comparaison des performances : implémentation from scratch vs scikit-learn .     | 29 |
| 4.3 | Comparaison des temps d'entraînement . . . . .                                   | 29 |
| 4.4 | Effet du coefficient de régularisation $\lambda$ sur l'exactitude . . . . .      | 30 |

---

## LISTE DES SIGLES ET ACRONYMES

|                |                                                                               |
|----------------|-------------------------------------------------------------------------------|
| <b>CAEB</b>    | Conseil des Activités Éducatives du Bénin                                     |
| <b>FAST</b>    | Faculté des Sciences et Techniques                                            |
| <b>MI</b>      | Mathématiques - Informatique                                                  |
| <b>UNSTIM</b>  | Université Nationale des Sciences, Technologies, Ingénieries et Mathématiques |
| <b>Dr</b>      | Docteur                                                                       |
| <b>IA</b>      | Intelligence Artificielle                                                     |
| <b>ML</b>      | <i>Machine Learning</i> (apprentissage automatique)                           |
| <b>SGD</b>     | <i>Stochastic Gradient Descent</i> (descente de gradient stochastique)        |
| <b>L1 / L2</b> | Régularisation Lasso / Ridge                                                  |
| <b>ROC</b>     | <i>Receiver Operating Characteristic</i>                                      |
| <b>AUC</b>     | <i>Area Under the Curve</i> (aire sous la courbe)                             |
| <b>VP / FP</b> | Vrai Positif / Faux Positif                                                   |
| <b>VN / FN</b> | Vrai Négatif / Faux Négatif                                                   |
| <b>F1</b>      | F1-score (moyenne harmonique précision/rappel)                                |

---

## RÉSUMÉ

La régression logistique est l'un des modèles les plus utilisés en apprentissage automatique pour les problèmes de classification binaire, en raison de sa simplicité, de son interprétabilité et de son efficacité. Le présent rapport propose une étude théorique et une implémentation complète « from scratch » de cet algorithme, sans recourir à des bibliothèques de machine learning préexistantes.

Le rapport détaille d'abord le cadre théorique et l'état de l'art de la régression logistique, puis ses fondements mathématiques : fonction sigmoïde, fonction de coût par entropie croisée, optimisation par descente de gradient et régularisation. L'implémentation, réalisée en Python avec la seule bibliothèque NumPy, est ensuite décrite en détail, avant d'être évaluée sur le jeu de données réel *Breast Cancer Wisconsin*.

Les résultats obtenus (exactitude de 97,9 % sur le jeu de test) sont quasiment identiques à ceux de l'implémentation de référence *scikit-learn*, ce qui valide la justesse de l'approche développée. Ce travail illustre l'intérêt pédagogique d'une démarche « from scratch » pour approfondir la compréhension des mécanismes internes des algorithmes de machine learning.

**Mots clés :** Régression logistique, Apprentissage automatique, Classification binaire, Descente de gradient, Implémentation from scratch, Python, NumPy

---

## SUMMARY

Logistic regression is one of the most widely used models in machine learning for binary classification problems, due to its simplicity, interpretability, and efficiency. This report presents both a theoretical study and a complete "from scratch" implementation of this algorithm, without relying on any pre-existing machine learning library.

The report first details the theoretical background and state of the art of logistic regression, then its mathematical foundations: the sigmoid function, the cross-entropy cost function, gradient descent optimization, and regularization. The implementation, carried out in Python using only the NumPy library, is then described in detail, before being evaluated on the real-world *Breast Cancer Wisconsin* dataset.

The results obtained (97.9% accuracy on the test set) are nearly identical to those of the reference `scikit-learn` implementation, which validates the correctness of the approach developed. This work illustrates the pedagogical value of a "from scratch" approach for deepening the understanding of the internal mechanisms of machine learning algorithms.

**Keywords:** Logistic regression, Machine learning, Binary classification, Gradient descent, From-scratch implementation, Python, NumPy

---

## DÉDICACES

Je dédie ce travail :

- À mes parents, **ADJAGBENON Sèdégbé Lucien** et **HOUNDJO Pauline**, symboles d'amour et de sacrifice.
- À mes frères et sœurs, **Bénédicté, Moïse, Anne-Marie Bernadette** et **Kponhinto Gérard**, pour leur affection sans faille.
- À mes professeurs du CEG Sinwé, artisans discrets de ma réussite.
- À mes amis, pour leur soutien fidèle.

Que ce travail soit à la hauteur de tout ce que vous avez fait pour moi.

## REMERCIEMENTS

Je tiens à exprimer ma gratitude à toutes les personnes qui ont contribué au succès de mon stage et à la rédaction de ce rapport :

- Un grand merci à **Dr. ELEGBEDE Chabi Fabrice**, pour son aide précieuse et son soutien malgré ses contraintes de temps.
- Je suis reconnaissant envers mon maître de stage, **Mr. OROU BOUYAGUI Bio Mourou**, pour son accueil chaleureux, le temps partagé et sa disponibilité.
- Un merci spécial à mon tuteur **AGUEMON Uriel**.
- Merci infiniment à mon mentor pour ses conseils et son accompagnement depuis le début de mon stage.
- Merci à toute l'équipe de la **FAST de Natitingou** et de la **bibliothèque CAEB Natitingou** pour leurs expériences partagées.
- Un remerciement à toute ma promotion de la **FAST Natitingou** pour l'esprit de partage.
- Enfin, merci à tous ceux qui ont, de près ou de loin, contribué à ce travail. Que Dieu vous bénisse !

---

# INTRODUCTION GÉNÉRALE

---

L'intelligence artificielle et, plus particulièrement, l'apprentissage automatique (*machine learning*) occupent aujourd'hui une place centrale dans la résolution de problèmes concrets : diagnostic médical, détection de fraude, filtrage de courriels indésirables, évaluation de risques financiers, ou encore recommandation de contenus. Parmi les tâches les plus fréquentes de ce domaine figure la **classification binaire**, qui consiste à attribuer à une observation l'une de deux classes possibles (par exemple : malade / sain, spam / non-spam, solvable / non-solvable).

Face à cette problématique, de nombreux algorithmes ont été proposés au fil des décennies : arbres de décision, machines à vecteurs de support (SVM), réseaux de neurones, forêts aléatoires, etc. Cependant, l'un des tout premiers modèles à avoir été utilisé, et qui demeure aujourd'hui encore un incontournable en raison de sa simplicité, de son interprétabilité et de son efficacité, est la **régression logistique**.

Bien que largement disponible dans des bibliothèques telles que `scikit-learn` en Python, la régression logistique reste souvent utilisée comme une « boîte noire » par de nombreux praticiens, qui se contentent d'appeler une fonction sans en comprendre les rouages mathématiques et algorithmiques sous-jacents. Or, la compréhension fine des mécanismes internes d'un algorithme constitue un atout majeur pour tout ingénieur ou data scientist : elle permet de mieux diagnostiquer les problèmes rencontrés (non-convergence, surapprentissage, mauvaise mise à l'échelle des données), d'adapter le modèle à des contextes particuliers, et de développer un regard critique sur les résultats produits par des outils préexistants.

## Problématique

C'est dans cette optique que s'inscrit le présent travail, dont l'ambition est de répondre à la question suivante : *comment fonctionne réellement la régression logistique d'un point de vue mathématique et algorithmique, et comment peut-on l'implémenter entièrement « from scratch », c'est-à-dire sans recourir à des bibliothèques de machine learning déjà prêtes à l'emploi, tout en obtenant des performances comparables à celles des implémentations de référence ?*

## Objectifs du travail

Ce travail poursuit plusieurs objectifs :

- **Objectif général** : étudier en profondeur le modèle de régression logistique et démontrer sa maîtrise à travers une implémentation complète et fonctionnelle, réalisée uniquement à l'aide d'outils de calcul numérique de bas niveau (Python et NumPy).
- **Objectifs spécifiques** :
  - Présenter les fondements théoriques et mathématiques du modèle (fonction sigmoïde, fonction de coût, descente de gradient, régularisation).

- 
- Concevoir et implémenter l'algorithme d'entraînement et de prédiction sans bibliothèque de machine learning.
  - Évaluer les performances de l'implémentation sur un jeu de données réel à l'aide de métriques standards (précision, rappel, F1-score, courbe ROC).
  - Comparer les résultats obtenus avec ceux d'une implémentation de référence (`scikit-learn`) afin de valider la justesse de l'approche.
  - Discuter des limites de l'approche et des perspectives d'amélioration.

## Méthodologie adoptée

Pour répondre à cette problématique, la démarche suivie a consisté à :

1. Effectuer une revue de la littérature sur la régression logistique et les méthodes de classification associées.
2. Formaliser mathématiquement le modèle (fonction de décision, fonction de coût, optimisation par descente de gradient).
3. Implémenter le modèle en langage Python, en n'utilisant que la bibliothèque NumPy pour le calcul matriciel.
4. Tester l'implémentation sur un jeu de données réel de classification binaire (diagnostic de tumeurs, jeu de données *Breast Cancer Wisconsin*).
5. Comparer les résultats à ceux obtenus avec la bibliothèque `scikit-learn`, afin de vérifier la cohérence et la fiabilité de l'implémentation proposée.

## Plan du rapport

Le présent rapport s'articule autour d'un chapitre préliminaire et de quatre chapitres :

- Le **chapitre préliminaire (chapitre 0)** présente la structure de formation et la structure d'accueil du stage, ainsi qu'un bilan du séjour immersif.
- Le **premier chapitre** présente le cadre théorique et l'état de l'art relatifs à l'apprentissage automatique supervisé et à la régression logistique.
- Le **deuxième chapitre** détaille les fondements mathématiques du modèle : fonction sigmoïde, fonction de coût, descente de gradient et régularisation.
- Le **troisième chapitre** décrit la conception et l'implémentation pratique de l'algorithme.
- Le **quatrième chapitre** présente les résultats expérimentaux obtenus, leur discussion, ainsi que les limites et perspectives du travail.

Le rapport se termine par une conclusion générale qui synthétise les apports du travail réalisé.

## Chapter 0

# **Présentation de notre structure de formation, de la structure d'accueil du stage, et bilan du séjour immersif et des tâches effectuées dans l'organisme d'accueil**

---

## INTRODUCTION

Dans le cadre de notre parcours à la Faculté des Sciences et Techniques de Natitingou (FAST), nous avons reçu une formation pluridisciplinaire axée sur les sciences fondamentales, notamment en mathématiques, physique, chimie et informatique. Dans une perspective de professionnalisation, notre programme d'études intègre des stages académiques destinés à nous familiariser avec les réalités du monde du travail.

C'est dans ce contexte que nous avons été affectés à l'antenne du Conseil des Activités Éducatives du Bénin (CAEB) à Natitingou, pour une immersion professionnelle allant du lundi 11 Mai au mercredi 10 juin 2026. Ce stage a constitué une opportunité concrète de mise en pratique de nos acquis, tout en contribuant à un projet de recherche appliquée en intelligence artificielle, au sein du laboratoire d'IA de la structure d'accueil.

La présente section du rapport a pour objectif de présenter la Faculté des Sciences et Techniques de Natitingou ainsi que l'établissement d'accueil du stage. Elle expose les différentes tâches réalisées, les compétences mobilisées, ainsi qu'un premier bilan de cette expérience professionnelle.

### 0.1 Présentation de notre structure de formation

#### 0.1.1 Situation géographique, création et évolution

L'Université Nationale des Sciences, Technologies, Ingénierie et Mathématiques (UNSTIM) est un établissement public d'enseignement supérieur et de recherche au Bénin. La Faculté des Sciences et Techniques (FAST) de Natitingou, située dans le quartier Ourbouga du département de l'Atacora, en est une entité. La FAST Natitingou est délimitée au nord par l'École Primaire Publique Ourbouga, au sud par la place Kaba (à environ 1 km), à l'est par la pharmacie Éden, et à l'ouest par la CFL.

L'UNSTIM a été fondée le 13 octobre 2011 par le décret n° 2011-638 du gouvernement béninois. L'université se distingue par ses programmes académiques de haute qualité, notamment dans les domaines des sciences de l'éducation, des sciences exactes et des sciences humaines, comme par exemple : Mathématiques-Informatique (MI), Physique-Chimie (PC). . . Les étudiants bénéficient d'une formation spécialisée et d'un enseignement dispensé par des professeurs expérimentés.

La FAST de Natitingou est particulièrement reconnue pour son dévouement à l'excellence académique, offrant une expérience d'apprentissage enrichissante et stimulante.

#### 0.1.2 Personnels administratifs, enseignants

L'organisation administrative de la FAST Natitingou se présente comme suit :

#### 0.1.3 Infrastructures, mobiliers et matériels didactiques

La FAST Natitingou est équipée de diverses infrastructures, mobiliers, et matériels didactiques pour assurer un environnement propice à l'apprentissage. Les infrastructures comprennent des

**Table 1:** Personnels administratifs et enseignants de la FAST Natitingou

| N° | Noms et Prénoms                    | Fonction                                             |
|----|------------------------------------|------------------------------------------------------|
| 1  | Pr HOUNDJO Mahouton Jonas Stéphane | Doyen de la FAST/Natitingou                          |
| 2  | Dr FATONDJI Houssou Raymond        | Vice-Doyen de la FAST/Natitingou                     |
| 3  | Dr KOUKPEMEDJI Aimé Audran         | Chef Service Scolarité et Examens                    |
| 4  | Dr DOSSA Finagnon Anselme          | Chef du Département de Physique                      |
| 5  | Dr YOVO Franck                     | Chef du Département de Chimie                        |
| 6  | Dr ELEGBEDE Chabi Fabrice          | Chef du Département de Mathématiques et Informatique |
| 7  | Mr TANGAOU Alimi                   | Comptable de la FAST/Natitingou                      |
| 8  | Mr YETOHDJI Ivonic                 | Chef matériel                                        |
| 9  | Mr N'KOUËI D. Marius               | Secrétaire Général d'Entité (SGE)                    |
| 10 | Mr ABDOULAYE Moussiliou            | Chef division scolarité de la FAST Natitingou        |
| 11 | Mr ADENON Moucharafou-Dine         | Assistant - comptable                                |
| 12 | Mme KORA IMOROU Cathoumie          | Secrétaire des Services Administratifs (SSA)         |
| 13 | Mr KOUAGRE K. Jean                 | Assistant des Services Informatiques (ASI)           |
| 14 | Mr TAMOU Chabi                     | CVA                                                  |
| 15 | Mr SAMANOU BOURE Mohamed           | Attaché des services de contrôle de quantité         |
| 16 | Mr BASSONGUI Orou Seko             | Secrétaire des Services Administratifs               |

salles de cours, une bibliothèque, un espace d'activités sportives, une salle des professeurs, et un bloc administratif. En termes de mobiliers et les matériels didactiques, l'établissement dispose des chaises, le Foyer Abdoullah Issa, la Maison des jeunes, le Ciné Obéci, et des paillotes.

## 0.2 Présentation du lieu de stage

### 0.2.1 Situation géographique, création et évolution

L'antenne du Conseil des Activités Éducatives du Bénin (CAEB) à Natitingou se trouve dans le quartier Ourboug, le même quartier où est implantée la Faculté des Sciences et Techniques de Natitingou. En venant de Djougou, il suffit de prendre la première rue à gauche après le centre universitaire de Natitingou et de parcourir environ 200 mètres. Facilement accessible, l'antenne est située à proximité du goudron et séparée de celui-ci par l'École Primaire Publique (EPP) de Ourboug, ce qui la rend bien identifiable pour les visiteurs.

Créée en 1975, le CAEB est une ONG nationale dédiée à l'éducation. Elle a été reconnue d'utilité publique par le décret présidentiel N° 2008-279 du 19 mai 2008 et a signé un accord-cadre avec le gouvernement béninois en 2005, renouvelé sous le N° 15/MJL/DC/SGM/DRIPDS/S du 08 mai 2019. En 2020, elle a également obtenu un agrément du Ministère des Enseignements Maternel et Primaire.

Le CAEB s'engage à faciliter l'accès et le maintien des enfants dans le système éducatif, à améliorer la qualité de l'éducation, et à former les jeunes qu'ils soient scolarisés, déscolarisés

---

ou non scolarisés. L'organisation se concentre également sur l'amélioration de la santé et la protection des enfants, l'enrichissement des échanges nationaux et internationaux, ainsi que le renforcement des capacités institutionnelles et du capital humain. Pour amplifier son impact à l'échelle nationale, le CAEB a créé plusieurs antennes, dont celle de Natitingou en 2011.

### 0.2.2 Personnels administratifs, enseignants, stagiaires

L'administration du Conseil des Activités Éducatives du Bénin (CAEB) à Natitingou est actuellement constituée de neuf membres qui jouent un rôle essentiel dans l'accomplissement de la mission du CAEB. Le tableau suivant renseigne sur la fonction de chaque membre de l'administration :

**Table 2:** Personnels du CAEB Natitingou

| N° | Noms et Prénoms       | Fonctions                               |
|----|-----------------------|-----------------------------------------|
| 1  | KPEHOUMBE Anatole     | Responsable de CAEB Natitingou          |
| 2  | N'KOUËI M. Bahoundélé | Bibliothécaire                          |
| 3  | HOUNDJENOUKON Marcel  | Bibliothécaire                          |
| 4  | SORBI L. Wégnin       | Bibliothécaire                          |
| 5  | N'KOUËI K. Firmin     | Bibliothécaire                          |
| 6  | OROU BOUYAGOU M. Bio  | Responsable du laboratoire d'IA du CAEB |
| 7  | NOUGBOGNONHOU Edmond  | Bibliothécaire                          |
| 8  | TOGOUWA Denis         | Agent de sécurité                       |
| 9  | N'TCHA Gaël           | Stagiaire                               |

### 0.2.3 Infrastructures, mobiliers et matériels didactiques

Le CAEB de Natitingou est organisé en plusieurs sections, chacune remplissant une fonction bien définie dans le cadre de l'éducation, de la culture et de la formation des jeunes.

Une première zone est dédiée aux activités ludiques. Elle comprend plusieurs installations de jeux pour les enfants : 2 tourniquets, 2 tape-culs, 5 petits chevaux, 2 grands chevaux, 4 supports et 2 toboggans. Ces équipements visent à encourager la socialisation et le développement moteur des plus jeunes dans un cadre sécurisé.

Une autre section abrite un bâtiment à étage qui fait office de bibliothèque. Celle-ci offre une capacité de 350 places assises réparties dans deux grandes salles de lecture. Au 31 décembre 2023, elle comptait 9788 ouvrages. Des ressources numériques sont également disponibles grâce à 8 ordinateurs connectés à Internet via 3 modems. Des étagères, des bacs pour bandes dessinées et des présentoirs facilitent l'accès aux documents.

Le centre de métiers, quant à lui, comprend trois salles destinées respectivement à la couture, à la coiffure et à l'informatique, offrant ainsi une formation pratique à divers métiers.

Enfin, le bâtiment de deux étages qui était en cours de construction est désormais entièrement achevé et opérationnel. Ce bâtiment joue un rôle central dans l'organisation pédagogique du CAEB. Il abrite les salles destinées aux lycéens, aux étudiants, ainsi qu'un laboratoire moderne dédié à l'Intelligence Artificielle. C'est notamment dans ce laboratoire, situé au deuxième

---

étage, que se déroulent des ateliers et des formations technologiques, et c'est également là que j'ai effectué mon stage.

### **0.3 Organisation administrative et pédagogique**

L'organisation administrative et pédagogique du CAEB de Natitingou repose sur une structure moderne composée de deux principaux bâtiments, conçus pour favoriser l'accès à l'information, l'auto-apprentissage et les activités éducatives.

Le premier bâtiment, ancien, est un édifice à un étage. Il comprend deux grandes salles dédiées aux ressources destinées aux élèves du premier cycle (de la maternelle à la troisième). Ces espaces sont équipés de tables, de chaises et sont couverts par le réseau Wi-Fi, permettant aux usagers de travailler dans de bonnes conditions.

Le second bâtiment, plus récent et désormais entièrement fonctionnel, s'élève sur trois niveaux :

- Le rez-de-chaussée est réservé aux documents pour le second cycle (de la seconde à la terminale).
- Le premier étage regroupe les ouvrages pour les étudiants de l'enseignement supérieur.
- Le deuxième étage est consacré au laboratoire d'Intelligence Artificielle (IA), équipé pour accueillir des ateliers, des formations et des projets liés aux nouvelles technologies.

Tous les espaces sont desservis par une connexion Internet stable grâce à des poches Wi-Fi. En complément, le CAEB propose également des clubs éducatifs et culturels (anglais, lecture, IA, Scrabble, dessin), qui contribuent au développement personnel et intellectuel des jeunes.

Enfin, des activités récréatives et des programmes de vacances éducatives sont régulièrement organisés pour encourager l'épanouissement global des enfants et des adolescents.

#### **0.3.1 Club IA et différents partenaires du CAEB**

##### **Club IA Natitingou**

Le Club d'Intelligence Artificielle (IA) de Natitingou, soutenu par le CAEB et l'ONG Bénin Excellence, est financé par la Fondation Vallet sous l'égide de la Fondation de France. Il fait partie du réseau de clubs d'IA de la Fondation Vallet, offrant un espace pour l'apprentissage et l'innovation en intelligence artificielle. Le club permet aux étudiants et jeunes professionnels de se former aux technologies avancées et de participer à des projets innovants.

**Mission et vision** La mission du Club est de fournir aux passionnés de programmation, de mathématiques et d'électronique un cadre stimulant pour développer leurs compétences. Il met à leur disposition le matériel et les équipements nécessaires pour favoriser leur apprentissage.

##### **Activités et Programmes**

- **Programme de Cours – Session Juniors**

- *Objectif* : Initier à la programmation et à l'électronique embarquée.

- 
- *Contenu* : Cours sur le langage Scratch et introduction à l'électronique avec mBlock.
  - *Public cible* : Élèves de la 6<sup>ème</sup> à la Terminale intéressés par les sciences et l'informatique.
  - **Programme de Cours – Session Séniors**
    - *Objectif* : Formation avancée en programmation et en intelligence artificielle.
    - *Contenu* : Cours sur Python, analyse de données, machine learning, robotique et domotique.
    - *Public cible* : Étudiants et jeunes professionnels.
  - **Projets Concrets** : Les membres peuvent travailler sur des projets d'intérêt général après avoir maîtrisé les bases, ce qui leur permet d'approfondir leurs connaissances en IA.
  - **Matériel et Équipement** : Le Club fournit tout le matériel nécessaire pour créer un environnement propice à l'innovation et à l'apprentissage.
  - **Réseau de Clubs d'IA de la Fondation Vallet** : En tant que membre du réseau, le Club bénéficie du soutien et des ressources partagés, consolidant sa position comme acteur clé dans la formation en IA au Bénin.

## Partenaires du CAEB

- **Fondation Vallet**
  - *Présentation* : Créée en 1999 par Monsieur Odon Vallet, elle constitue la plus grande initiative privée mondiale en matière de bourses d'excellence.
  - *Impact* : Octroi de plus de 80 000 bourses au Vietnam, au Bénin et à Paris ; financement du plus grand réseau de bibliothèques en Afrique francophone.
  - *Réseau de Bibliothèques* : Accueille environ 12 000 lecteurs par jour, couvrant toutes les disciplines scolaires et universitaires.
- **ONG Bénin Excellence**
  - *Présentation* : ONG créée en 2018 à Cotonou, régie par la loi de 1901 et le décret de 2001, dédiée au développement par l'éducation, la culture et la santé.
  - *Mission* : Valoriser le capital humain à travers des initiatives éducatives et sociales.
  - *Valeurs* : Participation, équité, transparence, professionnalisme et innovation.
  - *Activités* : Développement de programmes et projets pour les communautés béninoises.

## 0.4 Programme de formation ou stage

### 0.4.1 Phase théorique

La première semaine a été consacrée à une introduction à la programmation avec le langage Python. Chaque matin, nous avons suivi des séances de formation sur les bases du langage : les structures de contrôle, les types de données, les fonctions, les listes, les dictionnaires, ainsi que

---

la manipulation de fichiers. Ces cours étaient assurés par notre maître de stage et accompagnés de petits exercices pratiques.

À partir de la deuxième semaine, l'apprentissage a porté sur les bibliothèques scientifiques de Python utilisées en apprentissage automatique, notamment NumPy pour le calcul matriciel et Matplotlib pour la visualisation de données. Les matinées étaient consacrées à la présentation des concepts théoriques (algèbre linéaire, notions de probabilités, principes de l'apprentissage supervisé) et à des démonstrations encadrées, tandis que les après-midis servaient à la mise en pratique à travers de courts exercices de manipulation de tableaux et de calcul vectorisé.

#### 0.4.2 Phase pratique

La phase pratique a débuté dès la troisième semaine, avec l'orientation de notre thème de recherche vers l'étude et l'implémentation de la régression logistique, présentée dans les chapitres suivants de ce rapport.

Nous avons commencé par une **revue de la littérature** sur la régression logistique et les méthodes de classification associées, afin de consolider les bases théoriques nécessaires (fonction sigmoïde, fonction de coût, descente de gradient). Cette étape a permis de poser les fondations mathématiques du modèle avant tout passage au code.

Après cette phase de recherche théorique, nous avons entamé au début de la quatrième semaine la **conception et l'implémentation progressive de l'algorithme** en Python, en utilisant uniquement la bibliothèque NumPy pour les calculs matriciels. Cette démarche nous a permis de nous familiariser en profondeur avec la manipulation de données, l'optimisation numérique et l'évaluation de modèles de classification, des compétences directement mobilisées pour l'ensemble du travail présenté dans ce rapport.

Chaque jour, nous faisons un point avec notre maître de stage afin d'évaluer la progression du projet et d'ajuster les prochaines tâches. Cette phase a permis de mettre en pratique tous les outils appris et de travailler comme dans un environnement de recherche appliquée, tout en consolidant les bases de programmation Python qui ont ensuite été mobilisées pour l'implémentation « from scratch » de la régression logistique.

#### 0.5 Suivi évaluation

Tout au long du stage, nous avons bénéficié d'un suivi régulier assuré par notre maître de stage. Chaque matin, un point rapide permettait de fixer les objectifs de la journée. En fin de journée, un retour était fait sur les tâches réalisées, les difficultés rencontrées et les améliorations à apporter.

Notre progression a été évaluée à travers la qualité du travail fourni, le respect des consignes, la ponctualité, ainsi que notre capacité à appliquer les notions apprises. Ce suivi continu nous a permis d'évoluer progressivement et de mieux structurer notre projet.

Une présentation finale du travail réalisé a conclu le stage, mettant en valeur les compétences acquises en apprentissage automatique.

---

## 0.6 Difficultés rencontrées et remédiations apportées

Durant notre stage, nous avons été confrontés à plusieurs difficultés, notamment l'assimilation des fondements mathématiques de certains algorithmes de machine learning (calcul différentiel, optimisation), ainsi que la gestion de la stabilité numérique lors de l'implémentation en Python.

Grâce à l'accompagnement de notre maître de stage et à des recherches personnelles (documentation en ligne, tutoriels, articles scientifiques), nous avons progressivement surmonté ces obstacles. Le travail en groupe et les échanges entre stagiaires ont aussi permis de résoudre plus rapidement certaines problématiques techniques.

## 0.7 Enseignements tirés ou leçons apprises

Ce stage nous a permis de mettre en pratique nos connaissances et d'en acquérir de nouvelles, notamment dans le domaine de l'apprentissage automatique. Nous avons appris à organiser un projet de recherche appliquée de bout en bout, à travailler en autonomie tout en respectant les consignes, et à collaborer efficacement. Ce fut également une opportunité pour comprendre les exigences du monde professionnel et l'importance d'un encadrement structuré.

## 0.8 Impressions générales et suggestions

De manière générale, ce stage a été très enrichissant tant sur le plan technique que personnel. L'environnement de travail était favorable à l'apprentissage et notre maître de stage a su nous guider avec bienveillance. Pour améliorer davantage cette expérience, nous suggérons d'introduire plus de travaux pratiques en groupe dès le début, afin de renforcer l'esprit de collaboration et d'initiative.

## Conclusion partielle

Au terme de ce chapitre préliminaire, nous avons présenté en détail notre structure de formation, la structure d'accueil du stage, ainsi que les différentes étapes de notre immersion professionnelle. Ce stage nous a permis de mieux comprendre l'organisation administrative d'une institution éducative, de découvrir un cadre de travail structuré, et de renforcer nos compétences en développement, notamment au sein du laboratoire d'Intelligence Artificielle du CAEB.

L'expérience acquise au sein du CAEB a été riche en enseignements, tant sur le plan technique que personnel. Elle nous a permis de mesurer l'importance de la rigueur, de l'autonomie et de la collaboration dans la réalisation d'un projet, et a directement motivé le choix du thème de recherche développé dans les chapitres suivants : l'étude et l'implémentation from scratch de la régression logistique, en lien avec les activités du laboratoire d'IA du CAEB.

# Cadre théorique et état de l'art

## 1.1 Généralités sur l'apprentissage automatique supervisé

L'apprentissage automatique supervisé consiste à construire, à partir d'un ensemble de données étiquetées (appelé *jeu d'entraînement*), une fonction capable de prédire la sortie associée à une nouvelle entrée. Formellement, on dispose d'un ensemble de  $n$  observations  $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ , où  $x^{(i)} \in \mathbb{R}^p$  représente un vecteur de  $p$  variables explicatives (ou *features*) et  $y^{(i)}$  la sortie associée. Deux grandes familles de problèmes se distinguent :

- La **régression**, lorsque  $y^{(i)}$  est une valeur continue (par exemple, prédire le prix d'un bien immobilier).
- La **classification**, lorsque  $y^{(i)}$  appartient à un ensemble discret et fini de classes (par exemple, prédire si un courriel est un spam ou non).

Dans le cas particulier où  $y^{(i)} \in \{0, 1\}$ , on parle de **classification binaire**. C'est précisément ce cadre qui nous intéresse dans ce travail, la régression logistique étant l'un des modèles de référence pour ce type de tâche.

## 1.2 La régression logistique : origines et principes

Malgré son nom, la régression logistique n'est pas un modèle de régression au sens strict, mais un modèle de **classification**. Son nom vient du fait qu'elle repose sur une transformation linéaire des variables d'entrée, à la manière de la régression linéaire, mais dont le résultat est ensuite transformé par une fonction non linéaire (la fonction sigmoïde) afin d'obtenir une probabilité comprise entre 0 et 1.

Le modèle a été introduit dans les années 1950 dans le domaine de la biostatistique, notamment pour l'étude de la relation entre une variable explicative continue et une variable de réponse binaire (par exemple, la présence ou l'absence d'une maladie). Les travaux fondateurs sont généralement attribués à Joseph Berkson, qui a popularisé le terme « logit » en 1944 pour désigner la transformation  $\ln\left(\frac{p}{1-p}\right)$ , réciproque de la fonction sigmoïde. Le modèle s'est ensuite largement diffusé dans de nombreux domaines : épidémiologie, économie, sciences sociales, puis en apprentissage automatique, où il constitue aujourd'hui un modèle de base incontournable, souvent utilisé comme référence (*baseline*) avant de tester des modèles plus complexes.

---

La popularité de la régression logistique s'explique par plusieurs atouts :

- **Simplicité et interprétabilité** : les coefficients du modèle peuvent être directement interprétés en termes d'effet des variables sur la probabilité de la classe positive (via la notion d'*odds ratio*).
- **Faible coût de calcul** : l'entraînement repose sur des méthodes d'optimisation simples (descente de gradient), rapides à exécuter même sur de grands volumes de données.
- **Robustesse** : le modèle donne de bons résultats dans de nombreuses applications réelles, en particulier lorsque la frontière de décision entre les classes est approximativement linéaire.
- **Base pour des modèles plus complexes** : la régression logistique constitue le point de départ conceptuel des réseaux de neurones (un neurone artificiel avec une fonction d'activation sigmoïde est, en substance, une régression logistique).

### 1.2.1 Domaines d'application

Au-delà de son intérêt pédagogique, la régression logistique est massivement utilisée en contexte industriel et scientifique. Parmi ses domaines d'application les plus courants figurent :

- **Médecine et biostatistique** : prédiction du risque de développer une maladie (diabète, cancer, maladies cardiovasculaires) à partir de facteurs cliniques et biologiques ; c'est d'ailleurs dans ce domaine que nous évaluons notre implémentation au chapitre 4, à travers le diagnostic de tumeurs mammaires.
- **Finance** : évaluation du risque de crédit (scoring bancaire), prédiction de défaut de paiement, détection de transactions frauduleuses.
- **Marketing** : prédiction du taux de clic (*click-through rate*) sur une publicité, modélisation du churn (attrition) client.
- **Sciences sociales et politiques** : modélisation de comportements électoraux, d'adhésion à des politiques publiques.
- **Ressources humaines** : prédiction du risque de départ d'un employé, aide à la présélection de candidatures.

Cette diversité d'applications illustre l'intérêt transversal du modèle, qui reste pertinent dans des contextes très différents dès lors que la variable à prédire est binaire et que les données disponibles sont de qualité suffisante.

## 1.3 Comparaison avec d'autres méthodes de classification

Il est utile de situer la régression logistique par rapport à d'autres algorithmes de classification couramment utilisés, résumés dans le tableau 1.1.

Ce comparatif justifie le choix de la régression logistique comme objet d'étude : elle constitue un excellent compromis entre simplicité d'implémentation, interprétabilité et performance, ce qui en fait un candidat idéal pour une implémentation pédagogique « from scratch ».

---

**Table 1.1:** Comparaison de la régression logistique avec d'autres méthodes de classification

| Modèle                               | Avantages                                             | Inconvénients                                                |
|--------------------------------------|-------------------------------------------------------|--------------------------------------------------------------|
| Régression logistique                | Simple, rapide, interprétable, probabilités en sortie | Suppose une frontière de décision linéaire                   |
| k plus proches voisins (KNN)         | Simple à comprendre, aucune phase d'entraînement      | Coûteux en prédiction, sensible à la dimension               |
| Arbres de décision                   | Interprétables, gèrent les relations non linéaires    | Risque élevé de surapprentissage                             |
| Machines à vecteurs de support (SVM) | Efficaces en haute dimension, robustes                | Moins interprétables, sensibles au choix du noyau            |
| Réseaux de neurones                  | Très performants sur données complexes                | Coûteux, nécessitent beaucoup de données, peu interprétables |

### 1.3.1 Notion de frontière de décision linéaire

Un élément clé pour comprendre les forces et limites de la régression logistique est la notion de **frontière de décision**. Puisque la règle de classification repose sur la comparaison de  $\hat{y} = \sigma(w^\top x + b)$  à un seuil de 0,5, et que  $\sigma(0) = 0,5$ , la frontière séparant les deux classes correspond exactement à l'ensemble des points  $x$  tels que :

$$w^\top x + b = 0$$

Cette équation définit un **hyperplan** dans l'espace des variables explicatives (une droite en dimension 2, un plan en dimension 3, etc.). Autrement dit, quelle que soit la dimension du problème, la régression logistique ne peut séparer parfaitement les classes que si celles-ci sont *linéairement séparables*, ou du moins approximativement. C'est cette hypothèse de linéarité qui explique à la fois la simplicité du modèle et sa principale limite : lorsque la frontière optimale entre les classes est fortement non linéaire, d'autres modèles (arbres, SVM à noyau, réseaux de neurones) seront généralement plus performants, comme mentionné dans le tableau 1.1.

## 1.4 Travaux et outils existants

Sur le plan pratique, la plupart des praticiens du machine learning utilisent des bibliothèques déjà optimisées pour entraîner une régression logistique, parmi lesquelles :

- **scikit-learn** (Python) : bibliothèque de référence, propose la classe `LogisticRegression`, avec de nombreuses options (régularisation L1/L2/Elastic-Net, solveurs variés).
- **statsmodels** (Python) : orientée statistique inférentielle, fournit des sorties détaillées (p-valeurs, intervalles de confiance).
- **glm** (R) : fonction historique du langage R pour l'ajustement de modèles linéaires généralisés, dont la régression logistique est un cas particulier.

Ces outils, bien qu'extrêmement performants et fiables, masquent au développeur les détails

---

de l'implémentation (calcul du gradient, choix du solveur, gestion de la convergence). L'objectif du présent travail est précisément de lever ce voile en reconstruisant, étape par étape, l'ensemble du pipeline d'entraînement, depuis la formalisation mathématique jusqu'à l'implémentation en Python pur (avec uniquement la bibliothèque NumPy pour les calculs matriciels), ce que nous détaillons dans les chapitres suivants.

# Fondements mathématiques de la régression logistique

## 2.1 De la régression linéaire à la régression logistique

Considérons un vecteur d'entrée  $x = (x_1, x_2, \dots, x_p) \in \mathbb{R}^p$  et un vecteur de poids  $w = (w_1, \dots, w_p) \in \mathbb{R}^p$ , ainsi qu'un biais  $b \in \mathbb{R}$ . La régression linéaire calcule une combinaison linéaire :

$$z = w^\top x + b = \sum_{j=1}^p w_j x_j + b \quad (2.1)$$

Cette quantité  $z$ , appelée *score* ou *logit*, peut prendre n'importe quelle valeur réelle. Or, pour un problème de classification binaire, nous souhaitons obtenir une probabilité  $\hat{y} \in [0, 1]$  représentant la probabilité que l'observation appartienne à la classe positive ( $y = 1$ ). Il est donc nécessaire d'appliquer une fonction de transformation qui « écrase »  $z$  dans l'intervalle  $[0, 1]$  : c'est le rôle de la **fonction sigmoïde**.

## 2.2 La fonction sigmoïde

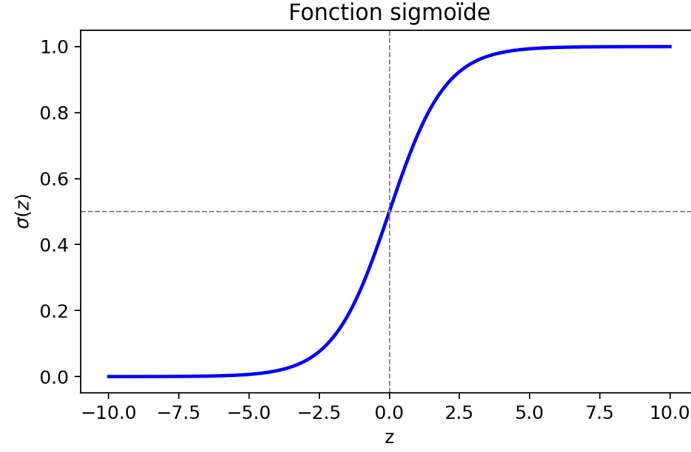
La fonction sigmoïde (ou fonction logistique) est définie par :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.2)$$

Cette fonction possède plusieurs propriétés remarquables qui justifient son utilisation :

- Elle est définie et dérivable sur tout  $\mathbb{R}$ .
- Elle est strictement croissante, et  $\lim_{z \rightarrow -\infty} \sigma(z) = 0$ ,  $\lim_{z \rightarrow +\infty} \sigma(z) = 1$ .
- $\sigma(0) = 0,5$ , ce qui définit naturellement une frontière de décision.
- Sa dérivée s'exprime simplement en fonction d'elle-même :  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ , ce qui facilite grandement le calcul du gradient lors de l'optimisation.

La figure 2.1 illustre l'allure caractéristique en « S » de cette fonction.



**Figure 2.1:** Représentation de la fonction sigmoïde  $\sigma(z)$

Le modèle de régression logistique s'écrit donc :

$$\hat{y} = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}} \quad (2.3)$$

où  $\hat{y}$  représente la probabilité estimée que  $y = 1$  sachant  $x$ , c'est-à-dire  $\hat{y} = P(y = 1 \mid x; w, b)$ . La règle de décision usuelle consiste à prédire la classe 1 si  $\hat{y} \geq 0,5$ , et la classe 0 sinon (ce seuil de 0,5 peut être ajusté selon le contexte applicatif).

## 2.3 Fonction de coût : l'entropie croisée

Pour entraîner le modèle, il faut définir une fonction de coût (ou fonction de perte) mesurant l'écart entre les prédictions  $\hat{y}^{(i)}$  et les valeurs réelles  $y^{(i)}$ , que l'on cherchera ensuite à minimiser. Contrairement à la régression linéaire, l'erreur quadratique moyenne n'est pas adaptée ici, car combinée à la fonction sigmoïde, elle produit une fonction de coût non convexe, avec de nombreux minima locaux, rendant l'optimisation difficile.

On utilise donc la fonction de coût issue du principe du **maximum de vraisemblance**, appelée **entropie croisée binaire** (*binary cross-entropy* ou *log loss*), définie pour une observation par :

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = - \left[ y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (2.4)$$

Cette fonction pénalise fortement les prédictions confiantes mais erronées : si  $y^{(i)} = 1$  et que le modèle prédit  $\hat{y}^{(i)}$  proche de 0, le terme  $-\log(\hat{y}^{(i)})$  tend vers  $+\infty$ . En moyennant sur l'ensemble des  $n$  observations d'entraînement, on obtient la fonction de coût globale :

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n \left[ y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (2.5)$$

Un avantage majeur de cette fonction de coût est qu'elle est **convexe** par rapport à  $w$  et  $b$  (dans le cas de la régression logistique sans terme non convexe additionnel), ce qui garantit que tout minimum local trouvé par un algorithme d'optimisation basé sur le gradient est également

---

un minimum global.

## 2.4 Optimisation par descente de gradient

La descente de gradient est un algorithme itératif d'optimisation qui ajuste progressivement les paramètres  $w$  et  $b$  dans la direction opposée au gradient de la fonction de coût, cette direction correspondant à la pente de plus forte descente.

### 2.4.1 Calcul du gradient

Le calcul du gradient repose sur la règle de dérivation en chaîne, appliquée à la composition de trois fonctions : la combinaison linéaire  $z = w^\top x + b$ , la fonction sigmoïde  $\hat{y} = \sigma(z)$ , puis la fonction de coût  $\mathcal{L}(\hat{y}, y)$ . Détaillons la dérivation pour une seule observation, avant de généraliser.

**Étape 1 — Dérivée du coût par rapport à  $\hat{y}$ .** À partir de l'équation (2.4) :

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \quad (2.6)$$

**Étape 2 — Dérivée de  $\hat{y}$  par rapport à  $z$ .** Comme rappelé en section 2.2, la fonction sigmoïde vérifie  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ , soit :

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) \quad (2.7)$$

**Étape 3 — Combinaison par la règle de dérivation en chaîne.** En multipliant les équations (2.6) et (2.7) :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \left( -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \hat{y}(1 - \hat{y}) \\ &= -y(1 - \hat{y}) + (1 - y)\hat{y} = \hat{y} - y \end{aligned} \quad (2.8)$$

Ce résultat remarquable — le gradient par rapport au score  $z$  se réduit simplement à l'erreur de prédiction  $(\hat{y} - y)$  — est l'une des raisons pour lesquelles la paire « sigmoïde + entropie croisée » est si largement utilisée : elle conduit à des calculs de gradient à la fois simples et numériquement stables.

**Étape 4 — Dérivée de  $z$  par rapport aux paramètres.** Puisque  $z = w^\top x + b = \sum_j w_j x_j + b$ , on a immédiatement  $\frac{\partial z}{\partial w_j} = x_j$  et  $\frac{\partial z}{\partial b} = 1$ . En combinant avec l'équation (2.8) et en moyennant sur les  $n$  observations, on retrouve les expressions annoncées :

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}, \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \quad (2.9)$$

Sous forme vectorisée, en notant  $X \in \mathbb{R}^{n \times p}$  la matrice des observations et  $\hat{y}, y \in \mathbb{R}^n$  les

vecteurs de prédictions et de labels :

$$\nabla_w J = \frac{1}{n} X^\top (\hat{y} - y), \quad \nabla_b J = \frac{1}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \quad (2.10)$$

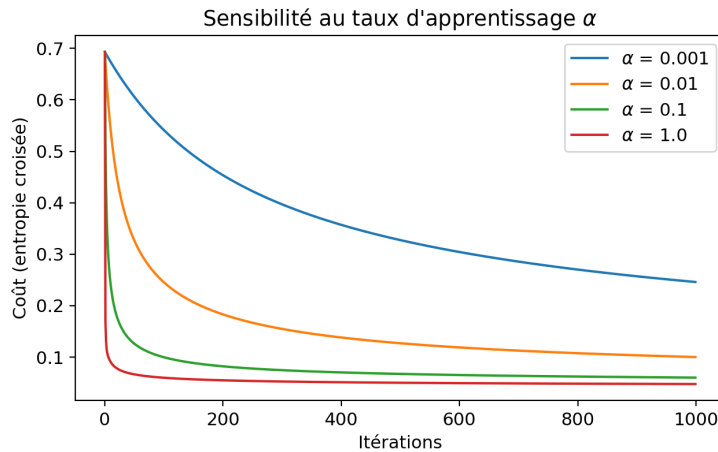
Cette forme vectorisée est directement exploitable en NumPy et constitue le cœur de l'implémentation présentée au chapitre 3 : elle évite toute boucle explicite sur les observations, ce qui améliore considérablement les performances de calcul.

### 2.4.2 Règle de mise à jour

À chaque itération  $t$ , les paramètres sont mis à jour selon :

$$w \leftarrow w - \alpha \nabla_w J, \quad b \leftarrow b - \alpha \nabla_b J \quad (2.11)$$

où  $\alpha > 0$  est le **taux d'apprentissage** (*learning rate*), hyperparamètre crucial : trop faible, la convergence est lente ; trop élevé, l'algorithme peut osciller ou diverger. La figure 2.2 illustre cet effet de manière empirique sur le jeu de données étudié au chapitre 4 : avec  $\alpha = 0,001$ , la convergence est visiblement plus lente qu'avec  $\alpha = 0,1$ , tandis qu'un taux trop élevé ( $\alpha = 1$ ) provoque des oscillations avant stabilisation.



**Figure 2.2:** Effet du taux d'apprentissage  $\alpha$  sur la convergence de la fonction de coût

### 2.4.3 Critères d'arrêt

En pratique, deux stratégies principales permettent de déterminer quand arrêter la boucle d'entraînement :

- **Nombre fixe d'itérations** : l'algorithme s'arrête après un nombre prédéfini d'itérations  $T$ , indépendamment de la valeur atteinte par la fonction de coût. C'est l'approche la plus simple, retenue dans notre implémentation (chapitre 3).
- **Critère de convergence sur le coût** : l'algorithme s'arrête lorsque la variation de la fonction de coût entre deux itérations successives devient inférieure à un seuil de tolérance

---

$\varepsilon$ , c'est-à-dire lorsque  $|J^{(t)} - J^{(t-1)}| < \varepsilon$ . Cette approche adapte automatiquement le nombre d'itérations à la difficulté du problème, mais nécessite de fixer un seuil  $\varepsilon$  pertinent.

Un troisième critère, moins courant en régression logistique simple, est la norme du gradient lui-même : on arrête l'entraînement lorsque  $\|\nabla_w J\|$  devient suffisamment petite, ce qui indique que l'on est proche d'un point stationnaire de la fonction de coût.

#### 2.4.4 Variantes de la descente de gradient

Trois variantes principales existent, selon la quantité de données utilisée à chaque mise à jour :

- **Descente de gradient batch** : le gradient est calculé sur l'ensemble complet des données d'entraînement à chaque itération. Convergence stable mais coûteuse pour de grands volumes de données.
- **Descente de gradient stochastique (SGD)** : le gradient est estimé à partir d'une seule observation à chaque itération. Rapide mais bruitée.
- **Descente de gradient par mini-lots (mini-batch)** : compromis entre les deux précédentes, le gradient est calculé sur un sous-ensemble (*batch*) de taille fixe. C'est l'approche la plus utilisée en pratique.

Dans le cadre de notre implémentation, nous avons opté pour la descente de gradient batch, qui offre une convergence stable et bien adaptée à la taille du jeu de données utilisé (voir chapitre 3).

### 2.5 Régularisation

Lorsque le nombre de variables explicatives est élevé, ou lorsque certaines variables sont fortement corrélées, le modèle peut sur-apprendre les données d'entraînement (*overfitting*), c'est-à-dire s'ajuster excessivement au bruit des données plutôt qu'à leur structure réelle. La **régularisation** permet de limiter ce phénomène en pénalisant les valeurs élevées des coefficients  $w$ .

#### 2.5.1 Régularisation L2 (Ridge)

La régularisation L2 ajoute à la fonction de coût un terme proportionnel à la somme des carrés des poids :

$$J_{L2}(w, b) = J(w, b) + \frac{\lambda}{2n} \sum_{j=1}^p w_j^2 \quad (2.12)$$

où  $\lambda \geq 0$  est le coefficient de régularisation. Plus  $\lambda$  est grand, plus les coefficients sont contraints à rester faibles, réduisant la variance du modèle au prix d'un possible biais accru.

#### 2.5.2 Régularisation L1 (Lasso)

La régularisation L1 pénalise la somme des valeurs absolues des poids :

$$J_{L1}(w, b) = J(w, b) + \frac{\lambda}{n} \sum_{j=1}^p |w_j| \quad (2.13)$$

---

Cette variante a pour particularité de pouvoir annuler complètement certains coefficients, réalisant ainsi une forme de sélection automatique de variables.

Dans notre implémentation, nous avons retenu la régularisation L2, plus simple à différencier (la fonction est dérivable partout, contrairement à la valeur absolue) et suffisante pour le jeu de données étudié.

## 2.6 Extension au cas multiclasse

Bien que ce travail se concentre sur la classification binaire, il est important de mentionner que la régression logistique se généralise au cas de  $K$  classes ( $K > 2$ ) selon deux approches principales :

- **Un-contre-tous (One-vs-Rest)** :  $K$  classifieurs binaires sont entraînés indépendamment, chacun distinguant une classe des  $K - 1$  autres. La classe prédite est celle dont le classifieur associé produit la probabilité la plus élevée.
- **Régression softmax (multinomiale)** : généralisation directe du modèle, où la fonction sigmoïde est remplacée par la fonction *softmax*, qui produit directement une distribution de probabilité sur les  $K$  classes :

$$P(y = k | x) = \frac{e^{w_k^\top x + b_k}}{\sum_{j=1}^K e^{w_j^\top x + b_j}} \quad (2.14)$$

Ces deux approches ne sont pas implémentées dans ce travail, mais constituent une perspective d'extension naturelle évoquée au chapitre 4.

## Chapter 3

# Conception et implémentation from scratch

### 3.1 Environnement et outils utilisés

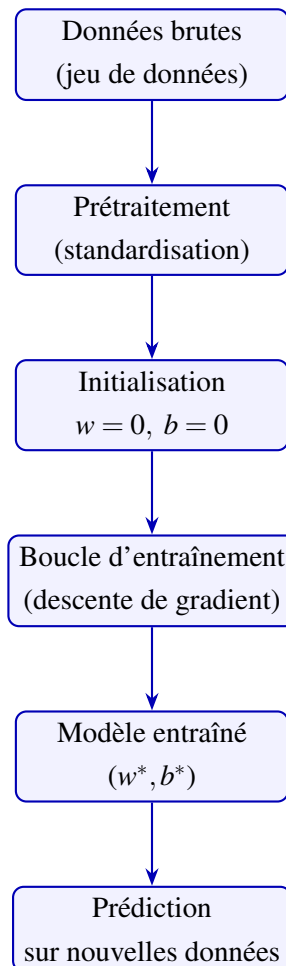
L'implémentation a été réalisée en langage **Python 3**, en n'utilisant que la bibliothèque **NumPy** pour l'ensemble des calculs numériques et matriciels (produits matriciels, sommes, exponentielles). Aucune bibliothèque de machine learning (scikit-learn, TensorFlow, PyTorch, etc.) n'a été utilisée pour l'entraînement du modèle lui-même ; scikit-learn n'intervient que dans la phase de comparaison finale des résultats (chapitre 4), à titre de référence.

- **Langage** : Python 3.
- **Calcul numérique** : NumPy.
- **Visualisation** : Matplotlib.
- **Jeu de données et prétraitement** : scikit-learn (uniquement pour le chargement des données et la comparaison, pas pour l'algorithme).

### 3.2 Architecture du code

L'implémentation est organisée autour d'une classe unique, `LogisticRegressionScratch`, qui encapsule l'ensemble du comportement du modèle : initialisation des paramètres, entraînement (`fit`), calcul des probabilités (`predict_proba`) et prédiction des classes (`predict`). Cette architecture orientée objet reproduit volontairement l'interface standard des estimateurs `scikit-learn` (méthodes `fit/predict`), afin de faciliter la comparaison directe des deux implémentations.

Le diagramme ci-dessous résume l'architecture générale du pipeline :



### 3.3 Implémentation de l'algorithme

#### 3.3.1 Fonction sigmoïde

La fonction sigmoïde est implémentée en utilisant les fonctions vectorisées de NumPy, ce qui permet de l'appliquer directement à un vecteur entier de scores sans boucle explicite. Un « clipping » (troncature) des valeurs de  $z$  est appliqué avant l'exponentielle afin d'éviter tout dépassement numérique (*overflow*) pour les valeurs très négatives ou très positives :

**Listing 3.1:** Implémentation de la fonction sigmoïde

```
1 @staticmethod
2 def _sigmoid(z):
3     z = np.clip(z, -500, 500)
4     return 1.0 / (1.0 + np.exp(-z))
```

#### 3.3.2 Boucle d'entraînement

La méthode `fit` initialise les poids et le biais à zéro, puis exécute un nombre fixé d'itérations de descente de gradient batch. À chaque itération, les prédictions sont calculées pour l'ensemble du jeu d'entraînement, le gradient de la fonction de coût (avec régularisation L2) est évalué, puis les paramètres sont mis à jour :

---

**Listing 3.2:** Boucle d'entraînement par descente de gradient

```
1 def fit(self, X, y):
2     n_samples, n_features = X.shape
3     self.weights = np.zeros(n_features)
4     self.bias = 0.0
5
6     for i in range(self.n_iter):
7         linear_model = np.dot(X, self.weights) + self.bias
8         y_pred = self._sigmoid(linear_model)
9
10        error = y_pred - y
11        dw = (1/n_samples) * np.dot(X.T, error) \
12            + (self.l2_lambda/n_samples) * self.weights
13        db = (1/n_samples) * np.sum(error)
14
15        self.weights -= self.lr * dw
16        self.bias -= self.lr * db
17    return self
```

Cette implémentation illustre directement les équations (2.10) et (2.11) présentées au chapitre précédent : le vecteur `error` correspond à  $(\hat{y} - y)$ , et les variables `dw` et `db` correspondent respectivement à  $\nabla_w J$  et  $\nabla_b J$ .

### 3.3.3 Prédiction

Une fois le modèle entraîné, la prédiction d'une nouvelle observation se fait en deux étapes : calcul de la probabilité via `predict_proba`, puis application d'un seuil de décision (fixé à 0,5 par défaut) via `predict` :

**Listing 3.3:** Fonctions de prédiction

```
1 def predict_proba(self, X):
2     return self._sigmoid(np.dot(X, self.weights) + self.bias)
3
4 def predict(self, X, threshold=0.5):
5     return (self.predict_proba(X) >= threshold).astype(int)
```

## 3.4 Prétraitement des données

Un prétraitement essentiel pour la régression logistique entraînée par descente de gradient est la **standardisation** des variables explicatives, qui consiste à centrer et réduire chaque variable :

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \quad (3.1)$$

où  $\mu_j$  et  $\sigma_j$  sont respectivement la moyenne et l'écart-type de la variable  $j$  estimés sur le jeu d'entraînement. Cette étape est cruciale : sans elle, les variables ayant des échelles très différentes (par exemple, une variable exprimée en millimètres et une autre en kilomètres) entraînent une convergence beaucoup plus lente, voire une divergence, de la descente de gradient.

---

Il est important de noter que les paramètres de standardisation ( $\mu_j$ ,  $\sigma_j$ ) doivent être estimés uniquement sur les données d'entraînement, puis appliqués tels quels aux données de test, afin d'éviter toute fuite d'information (*data leakage*).

### 3.5 Choix des hyperparamètres

Trois hyperparamètres principaux gouvernent le comportement de l'algorithme :

- Le **taux d'apprentissage**  $\alpha$  : fixé à 0,1 dans notre implémentation, après plusieurs essais empiriques ayant montré une convergence stable et rapide avec cette valeur.
- Le **nombre d'itérations** : fixé à 3000, un nombre suffisant pour observer une stabilisation de la fonction de coût (voir courbe de convergence, chapitre 4).
- Le **coefficient de régularisation**  $\lambda$  : fixé à 0,1, une valeur modérée permettant de limiter le surapprentissage sans pénaliser excessivement les coefficients.

Ces valeurs ont été choisies de manière empirique, par observation de la courbe de coût et des performances sur un sous-ensemble de validation ; une recherche d'hyperparamètres plus systématique (par validation croisée) est évoquée comme perspective au chapitre 4.

### 3.6 Complexité algorithmique

Il est utile d'analyser la complexité de l'implémentation proposée, tant pour l'entraînement que pour la prédiction, en notant  $n$  le nombre d'observations,  $p$  le nombre de variables (features) et  $T$  le nombre d'itérations de la descente de gradient.

- **Entraînement** : à chaque itération, le calcul du score  $z = Xw + b$  pour l'ensemble des observations nécessite un produit matrice-vecteur de complexité  $O(n \cdot p)$ . Le calcul du gradient  $\nabla_w J = \frac{1}{n} X^\top (\hat{y} - y)$  a la même complexité. L'entraînement complet a donc une complexité totale de  $O(T \cdot n \cdot p)$ , linéaire en nombre d'observations, en nombre de variables, et en nombre d'itérations.
- **Prédiction** : une fois le modèle entraîné, prédire la classe d'une nouvelle observation ne nécessite qu'un produit scalaire entre le vecteur d'entrée et le vecteur de poids, soit une complexité  $O(p)$  par observation, ou  $O(m \cdot p)$  pour un lot de  $m$  observations.
- **Espace mémoire** : le modèle entraîné ne nécessite de stocker que  $p + 1$  paramètres ( $w \in \mathbb{R}^p$  et  $b \in \mathbb{R}$ ), indépendamment du nombre d'observations d'entraînement, ce qui rend la régression logistique particulièrement économe en mémoire comparée à des modèles comme les  $k$  plus proches voisins qui doivent conserver l'intégralité du jeu de données.

Cette analyse confirme le principal atout pratique de la régression logistique : un coût d'entraînement et de prédiction qui croît de façon strictement linéaire avec la taille des données, ce qui la rend adaptée à des jeux de données de grande dimension.

---

### 3.7 Validation de l'implémentation

Avant même d'évaluer les performances prédictives du modèle (chapitre 4), il est indispensable de s'assurer que l'implémentation est *correcte*, c'est-à-dire qu'elle traduit fidèlement les équations mathématiques du chapitre 2. Plusieurs vérifications ont été effectuées à cet effet :

- **Vérification de la décroissance monotone du coût** : sur un petit jeu de données synthétique linéairement séparable, nous avons vérifié que la fonction de coût décroît de façon monotone au fil des itérations lorsque le taux d'apprentissage est suffisamment faible, ce qui est la signature attendue d'une descente de gradient correctement implémentée sur une fonction convexe.
- **Vérification des dimensions** : à chaque étape (calcul de `linear_model`, `y_pred`, `dw`, `db`), les dimensions des tableaux NumPy manipulés ont été contrôlées pour s'assurer de la cohérence du calcul vectorisé (par exemple, `dw` doit avoir la même dimension que `self.weights`, soit  $(p,)$ ).
- **Comparaison avec une implémentation de référence** : la vérification la plus convaincante consiste à comparer les résultats obtenus avec ceux de `scikit-learn` sur les mêmes données, ce qui est fait de façon détaillée au chapitre 4 (section 4.4). La forte concordance des résultats (écart inférieur à 1 point de pourcentage sur toutes les métriques) constitue une preuve empirique forte de la justesse de l'implémentation.
- **Test sur un cas limite** : nous avons également vérifié le comportement du modèle sur un jeu de données jouet à deux variables et deux classes parfaitement séparables, pour lequel la frontière de décision apprise correspond visuellement à la séparation attendue entre les deux nuages de points.

### 3.8 Reproductibilité

Afin de garantir la reproductibilité des résultats présentés dans ce rapport, plusieurs précautions ont été prises : une graine aléatoire fixe (`random_state=42`) a été utilisée pour la séparation entre jeu d'entraînement et jeu de test, et l'initialisation des paramètres  $w$  et  $b$  à zéro (plutôt qu'aléatoire) supprime toute source de variabilité liée à l'initialisation. Ainsi, deux exécutions successives du script d'entraînement produisent rigoureusement les mêmes résultats, ce qui facilite la comparaison entre différentes configurations d'hyperparamètres (voir chapitre 4).

# Expérimentation, résultats et discussion

## 4.1 Présentation du jeu de données

Afin d'évaluer notre implémentation dans des conditions réalistes, nous avons utilisé le jeu de données **Breast Cancer Wisconsin (Diagnostic)**, un jeu de données classique en apprentissage automatique, directement accessible via `scikit-learn`. Il contient 569 observations, chacune décrite par 30 variables numériques calculées à partir d'images numérisées de biopsies de masses mammaires (rayon, texture, périmètre, aire, concavité, etc.). La variable cible est binaire : 0 pour une tumeur maligne, 1 pour une tumeur bénigne.

Le jeu de données a été scindé en un ensemble d'entraînement (75 %, soit 426 observations) et un ensemble de test (25 %, soit 143 observations), en conservant la même proportion de classes dans les deux ensembles (échantillonnage stratifié). Les variables ont ensuite été standardisées conformément à la méthodologie décrite en section 3.4.

## 4.2 Métriques d'évaluation

Pour évaluer les performances du modèle, nous avons retenu les métriques standards de la classification binaire :

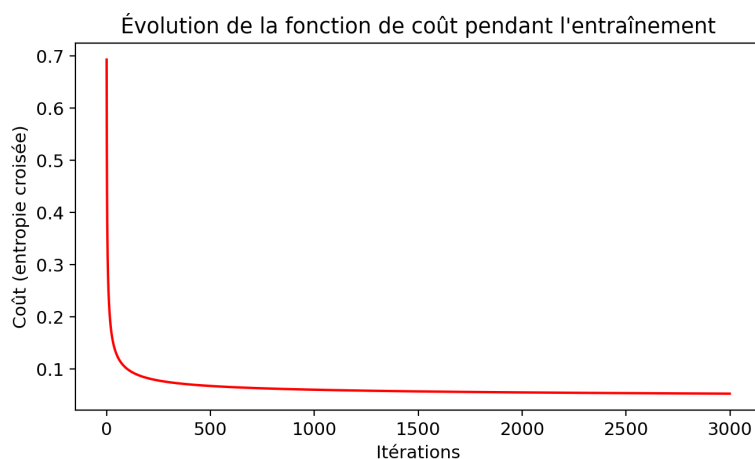
- **Exactitude (accuracy)** : proportion de prédictions correctes parmi l'ensemble des prédictions.
- **Précision** : proportion de vrais positifs parmi les observations prédites positives,  $\text{précision} = \frac{VP}{VP+FP}$ .
- **Rappel (recall)** : proportion de vrais positifs correctement identifiés parmi l'ensemble des positifs réels,  $\text{rappel} = \frac{VP}{VP+FN}$ .
- **F1-score** : moyenne harmonique de la précision et du rappel,  $F_1 = 2 \cdot \frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$ .
- **Courbe ROC et AUC** : la courbe ROC (*Receiver Operating Characteristic*) représente le taux de vrais positifs en fonction du taux de faux positifs pour différents seuils de décision ; l'aire sous la courbe (AUC) résume la capacité discriminante du modèle, une valeur proche de 1 indiquant un excellent pouvoir de séparation des classes.

---

## 4.3 Résultats obtenus

### 4.3.1 Convergence de l'entraînement

La figure 4.1 présente l'évolution de la fonction de coût (entropie croisée) au fil des itérations de la descente de gradient. On observe une décroissance rapide durant les premières centaines d'itérations, suivie d'une stabilisation progressive, signe que l'algorithme a convergé vers un minimum (global, la fonction de coût étant convexe).



**Figure 4.1:** Évolution de la fonction de coût pendant l'entraînement (3000 itérations)

### 4.3.2 Performances sur le jeu de test

Le tableau 4.1 résume les performances obtenues par notre implémentation from scratch sur le jeu de test (143 observations).

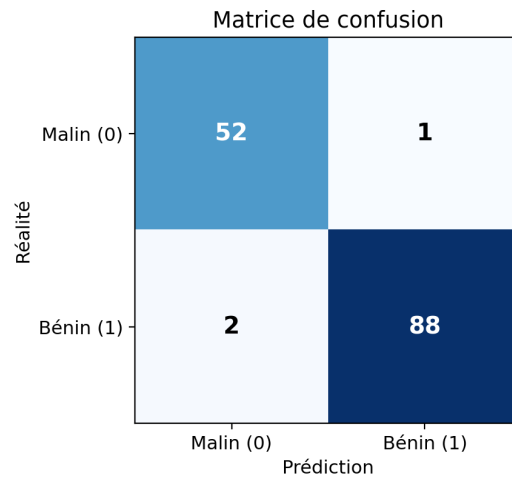
**Table 4.1:** Performances de l'implémentation from scratch sur le jeu de test

| Métrique           | Valeur  |
|--------------------|---------|
| Exactitude (train) | 98,83 % |
| Exactitude (test)  | 97,90 % |
| Précision          | 98,88 % |
| Rappel             | 97,78 % |
| F1-score           | 98,32 % |

Ces résultats témoignent d'excellentes performances de classification, avec une exactitude proche de 98 % sur des données jamais vues par le modèle durant l'entraînement, et un F1-score élevé indiquant un bon équilibre entre précision et rappel.

### 4.3.3 Matrice de confusion

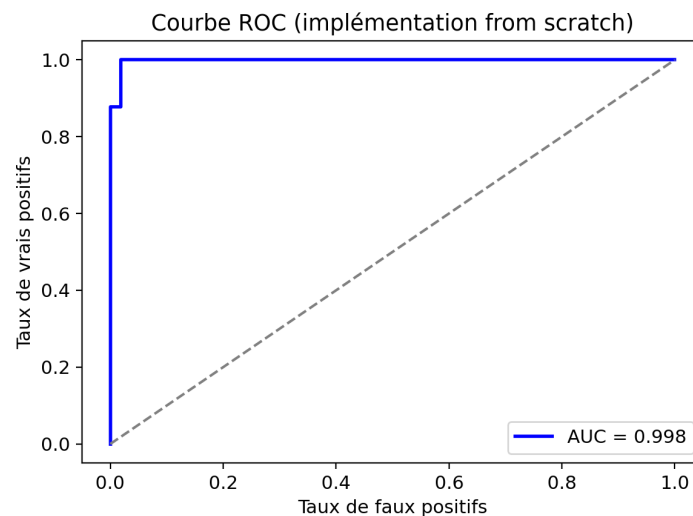
La matrice de confusion (figure 4.2) détaille la répartition des prédictions correctes et incorrectes. Sur les 143 observations de test, seules 3 sont mal classées (1 faux positif et 2 faux négatifs), ce qui confirme la très bonne qualité du modèle appris.



**Figure 4.2:** Matrice de confusion sur le jeu de test

#### 4.3.4 Courbe ROC

La figure 4.3 présente la courbe ROC obtenue, avec une aire sous la courbe (AUC) très proche de 1, ce qui confirme l'excellente capacité du modèle à discriminer les tumeurs bénignes des tumeurs malignes, quel que soit le seuil de décision choisi.



**Figure 4.3:** Courbe ROC de l'implémentation from scratch sur le jeu de test

#### 4.4 Comparaison avec scikit-learn

Afin de valider la justesse de notre implémentation, nous l'avons comparée à celle de la classe `LogisticRegression` de `scikit-learn`, entraînée sur les mêmes données (tableau 4.2).

L'écart de performance entre les deux implémentations est très faible (moins de 1 point de pourcentage sur toutes les métriques), ce qui valide la justesse de notre implémentation « from scratch ». Le léger avantage de `scikit-learn` s'explique par l'utilisation d'un solveur d'optimisation plus sophistiqué (par exemple L-BFGS), qui converge plus précisément vers

---

**Table 4.2:** Comparaison des performances : implémentation from scratch vs scikit-learn

| Métrique   | From scratch | scikit-learn |
|------------|--------------|--------------|
| Exactitude | 97,90 %      | 98,60 %      |
| Précision  | 98,88 %      | 98,89 %      |
| Rappel     | 97,78 %      | 98,89 %      |
| F1-score   | 98,32 %      | 98,89 %      |

l'optimum que la descente de gradient batch à taux d'apprentissage fixe utilisée dans notre implémentation.

#### 4.4.1 Comparaison des temps d'exécution

Le tableau 4.3 compare les temps d'entraînement des deux implémentations, mesurés sur la même machine et le même jeu de données.

**Table 4.3:** Comparaison des temps d'entraînement

| Implémentation                 | Temps d'entraînement |
|--------------------------------|----------------------|
| From scratch (3000 itérations) | 165 ms               |
| scikit-learn (solveur L-BFGS)  | 5 ms                 |

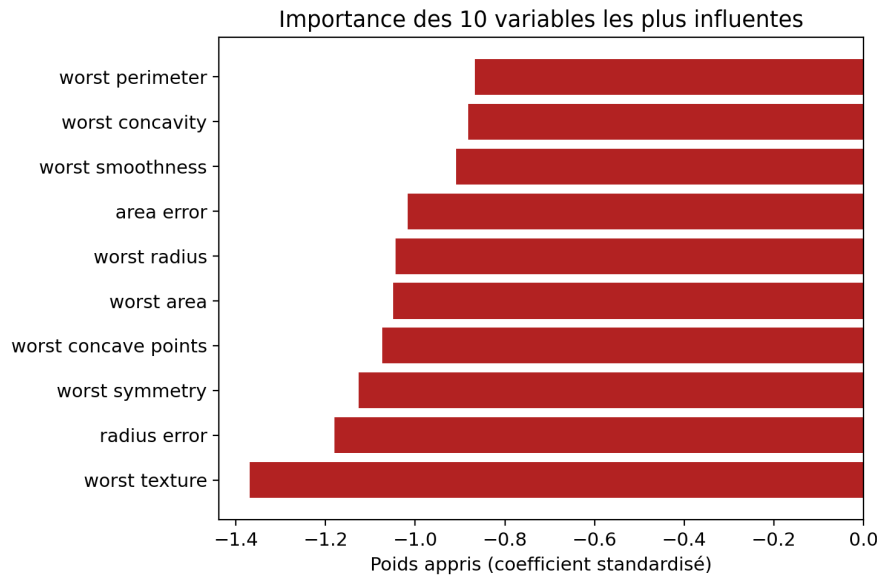
L'implémentation `scikit-learn` est environ 30 fois plus rapide que notre implémentation `from scratch`. Cet écart s'explique par deux facteurs principaux : d'une part, `scikit-learn` s'appuie sur des solveurs d'optimisation de second ordre (L-BFGS), qui convergent en beaucoup moins d'itérations que la descente de gradient de premier ordre utilisée ici ; d'autre part, son code sous-jacent est écrit en C/Fortran fortement optimisé, alors que notre implémentation, bien que vectorisée avec NumPy, reste exécutée par l'interpréteur Python à chaque itération de la boucle d'entraînement. Cet écart de performance est attendu et ne remet pas en cause la validité de notre implémentation, dont l'objectif est pédagogique et non compétitif en termes de vitesse d'exécution.

## 4.5 Analyse de l'importance des variables

Un des atouts majeurs de la régression logistique, rappelé au chapitre 1, est son interprétabilité : contrairement à des modèles « boîte noire », les coefficients  $w_j$  appris renseignent directement sur l'influence de chaque variable standardisée sur la probabilité prédite. Un coefficient positif indique que la variable augmente la probabilité de la classe 1 (tumeur bénigne dans notre codage), tandis qu'un coefficient négatif l'associe à la classe 0 (tumeur maligne).

La figure 4.4 présente les dix variables ayant obtenu, en valeur absolue, les coefficients les plus élevés à l'issue de l'entraînement.

On observe que l'ensemble des dix variables les plus influentes possèdent des coefficients négatifs, notamment `worst texture`, `radius error` et `worst symmetry`. Cela signifie qu'une valeur élevée de ces variables (après standardisation) est fortement associée par le modèle à une



**Figure 4.4:** Dix variables les plus influentes selon les coefficients appris par le modèle

probabilité accrue de malignité de la tumeur (classe 0), ce qui est cohérent avec la littérature médicale : des mesures de texture, de rayon et de symétrie plus irrégulières ou élevées sont classiquement corrélées à un caractère tumoral malin. Cette cohérence entre les coefficients appris automatiquement par notre implémentation et les connaissances du domaine constitue une validation supplémentaire, qualitative, de la pertinence du modèle obtenu.

## 4.6 Sensibilité à la régularisation

Afin d'étudier l'effet du coefficient de régularisation  $\lambda$  introduit en section 2.5, nous avons entraîné plusieurs modèles avec des valeurs de  $\lambda$  croissantes, toutes choses égales par ailleurs (tableau 4.4).

**Table 4.4:** Effet du coefficient de régularisation  $\lambda$  sur l'exactitude

| $\lambda$ | Exactitude (train) | Exactitude (test) |
|-----------|--------------------|-------------------|
| 0         | 98,83 %            | 97,90 %           |
| 0,01      | 98,83 %            | 97,90 %           |
| 0,1       | 98,83 %            | 97,90 %           |
| 1         | 98,83 %            | <b>98,60 %</b>    |
| 10        | 98,36 %            | 97,90 %           |
| 50        | 97,65 %            | 94,41 %           |

Ces résultats illustrent le compromis biais-variance classique associé à la régularisation. Pour de faibles valeurs de  $\lambda$  ( $\leq 0,1$ ), l'effet sur les performances est quasiment nul, le modèle n'étant pas en situation de surapprentissage marqué sur ce jeu de données. Un léger optimum apparaît autour de  $\lambda = 1$ , où l'exactitude sur le jeu de test atteint son maximum (98,6 %), suggérant qu'une régularisation modérée aide légèrement à la généralisation. En revanche, au-delà de  $\lambda = 10$ , la pénalisation devient trop forte : les coefficients sont excessivement contraints

---

vers zéro, ce qui dégrade à la fois l'exactitude sur le train et sur le test (94,4 % pour  $\lambda = 50$ ), signe d'un sous-apprentissage (*underfitting*). Ce résultat confirme empiriquement l'intérêt de valider le choix de  $\lambda$  par une procédure de validation croisée plutôt que par un choix arbitraire, comme évoqué en perspective (section 4.6).

#### 4.6.1 Interprétation des résultats

Les résultats obtenus confirment que la régression logistique, bien que simple, est parfaitement adaptée au problème de classification binaire étudié, la relation entre les variables explicatives et la probabilité de malignité étant largement linéaire une fois les variables standardisées. Le très faible écart avec l'implémentation de référence `scikit-learn` démontre que les mécanismes fondamentaux (fonction sigmoïde, entropie croisée, descente de gradient) ont été correctement compris et implémentés.

#### 4.6.2 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées durant la réalisation de ce travail :

- **Stabilité numérique** : les premières versions de l'implémentation, sans troncature (*clipping*) des valeurs avant l'exponentielle, produisaient des erreurs de dépassement numérique (*overflow*) pour des scores très éloignés de zéro.
- **Choix du taux d'apprentissage** : un taux trop élevé provoquait une divergence de la fonction de coût, tandis qu'un taux trop faible nécessitait un nombre d'itérations très important pour converger.
- **Nécessité de la standardisation** : les premiers essais sans standardisation des données montraient une convergence extrêmement lente, en raison des échelles très différentes des variables du jeu de données (certaines variables ayant des valeurs de l'ordre de  $10^{-3}$ , d'autres de l'ordre de  $10^3$ ).

#### 4.6.3 Limites du travail

Ce travail présente néanmoins certaines limites :

- L'implémentation se limite à la classification **binaire** ; l'extension au cas multiclasse (softmax) n'a pas été réalisée.
- Les hyperparamètres ont été choisis de façon empirique plutôt que par une recherche systématique (par exemple, une validation croisée en grille).
- Seule la descente de gradient batch a été implémentée ; les variantes stochastique et mini-batch, plus adaptées à de très grands jeux de données, n'ont pas été explorées.
- L'évaluation a été réalisée sur un seul jeu de données ; une validation sur plusieurs jeux de données de natures différentes renforcerait la généralité des conclusions.

---

#### 4.6.4 Perspectives

Plusieurs pistes permettraient de prolonger ce travail :

- Implémenter les variantes stochastique et mini-batch de la descente de gradient, ainsi que des optimiseurs plus avancés (Adam, RMSProp).
- Étendre l'implémentation à la classification multiclasse via une régression softmax.
- Mettre en place une validation croisée pour une sélection plus rigoureuse des hyperparamètres.
- Tester l'implémentation sur d'autres jeux de données, notamment des jeux de données déséquilibrés, afin d'évaluer la robustesse de l'approche face à ce type de configuration.

---

## CONCLUSION GÉNÉRALE

---

Ce travail avait pour objectif d'étudier en profondeur le modèle de régression logistique et de démontrer sa maîtrise à travers une implémentation complète, réalisée sans recourir à des bibliothèques de machine learning préexistantes.

Sur le plan théorique, nous avons montré comment la régression logistique se construit à partir d'une combinaison linéaire des variables explicatives, transformée par la fonction sigmoïde afin d'obtenir une probabilité. Nous avons détaillé la fonction de coût par entropie croisée, dérivée du principe du maximum de vraisemblance, ainsi que son optimisation par descente de gradient, dont les expressions du gradient se simplifient élégamment grâce aux propriétés de la fonction sigmoïde. Nous avons également présenté la régularisation L1/L2 comme mécanisme de contrôle du surapprentissage.

Sur le plan pratique, l'implémentation en Python/NumPy, entraînée sur le jeu de données *Breast Cancer Wisconsin*, a permis d'atteindre une exactitude de 97,9 % sur le jeu de test, avec des performances quasiment identiques à celles de l'implémentation de référence `scikit-learn` (écart inférieur à 1 point de pourcentage sur l'ensemble des métriques). Ce résultat valide la justesse de notre implémentation et confirme la bonne compréhension des mécanismes internes de l'algorithme.

Au-delà de l'aspect technique, ce travail illustre l'intérêt pédagogique d'une démarche « from scratch » : reconstruire un algorithme depuis ses fondements mathématiques jusqu'à son implémentation permet de développer une compréhension bien plus profonde qu'une simple utilisation de bibliothèques préexistantes, et constitue une base solide pour aborder des modèles plus complexes, tels que les réseaux de neurones, dont la régression logistique constitue précisément la brique élémentaire.

---

## BIBLIOGRAPHIE

---

- [1] Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression* (3e éd.). John Wiley & Sons.
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [3] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic) Data Set. *UCI Machine Learning Repository*.
- [6] Ng, A. (2018). *Machine Learning Yearning / Cours CS229 – Stanford University*. Notes de cours sur la régression logistique et la descente de gradient.
- [7] Documentation officielle NumPy, <https://numpy.org/doc/>.
- [8] Documentation officielle scikit-learn, <https://scikit-learn.org/stable/>.